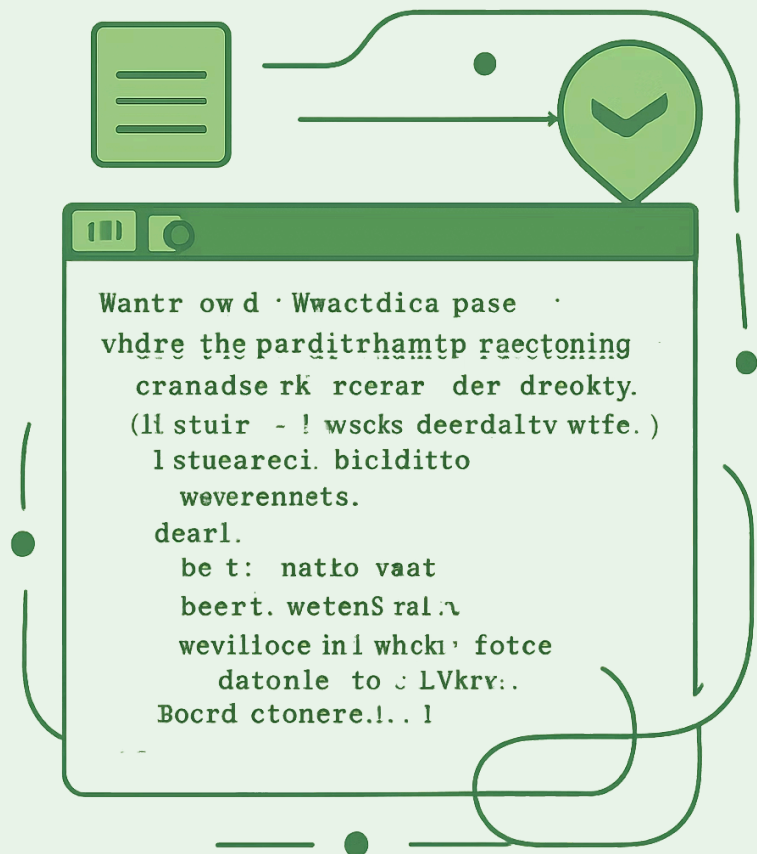
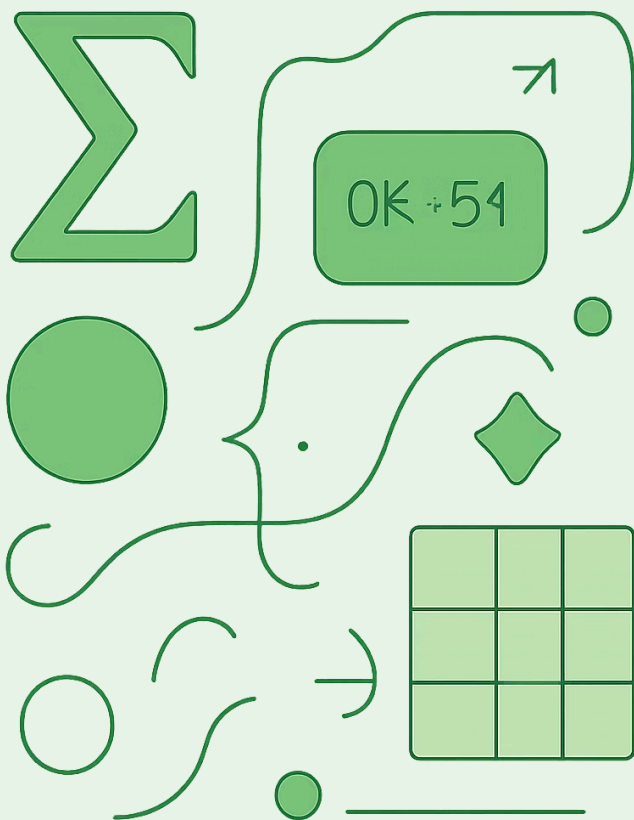




กลุ่มที่ 4

EXPRESSION PROCESSOR (INFIX และ POSTFIX)

รายงานฉบับนี้จัดทำขึ้นใหม่จากใบงานและรายงานกลุ่มเดิม ครอบคลุมการวิเคราะห์ปัญหา การออกแบบอัลกอริทึม 2 รูปแบบ การพิสูจน์ความถูกต้อง การวิเคราะห์ประสิทธิภาพ และการพัฒนาโปรแกรม Java พร้อมการทดสอบและทดลองประสิทธิภาพ



ที่มาและความสำคัญของปัญหา

นิพจน์ **Infix** กับคอมไพเลอร์

นิพจน์ทางคณิตศาสตร์ที่มนุษย์เขียนทั่วไปอยู่ในรูป Infix เช่น $3 + 4 * 2$ โดย Operator อยู่ระหว่าง Operand แต่คอมไพเลอร์ต้องพิจารณา **Operator Precedence** และ **Parentheses** ก่อนคำนวณ จึงเหมาะที่จะใช้ **Stack** ช่วยจัดลำดับการประมวลผล

เป้าหมายของระบบ

Expression Processor รับนิพจน์ Infix ตรวจสอบความถูกต้อง แปลง Infix เป็น Postfix และคำนวณผลลัพธ์ พร้อมเปรียบเทียบกับวิธีการประเมิน Infix โดยตรงด้วย Stack สองตัว

ขอบเขตงาน

- Operator: $+$ $-$ $*$ $/$ และ Parentheses $()$
- Operand: จำนวนเต็มบวกและจำนวนหลายหลัก
- รองรับช่องว่างระหว่าง Token
- ตรวจสอบวงเล็บไม่สมดุล, Operator ผิดตำแหน่ง, Division by Zero และ Empty Expression
- ใช้ Integer Division ตาม Java `int / int`
- ไม่บังคับรองรับทศนิยม, Unary Minus, ตัวแปร และฟังก์ชัน

การวิเคราะห์ปัญหา

Input / Output

Input: String ของ Infix เช่น "3 + 4 * 2 / (1 - 5)"

Output: Token list, Validation result, Postfix (Algorithm A) และ Integer result

Assumptions: การหารเป็น Integer Division, ไม่รองรับทศนิยม/Unary Minus/ตัวแปร

Edge Cases ที่ต้องจัดการ

- Empty expression "" → Validation Error
- วงเล็บไม่สมดุล (3 + 4 หรือ 3 + 4) → Validation Error
- Operator ผิดตำแหน่ง 3 + * 4 → Validation Error
- Division by zero 10 / (5 - 5) → ArithmeticException

Operation ที่เกิดบ่อย

การอ่าน Token และการ push/pop ของ Operand และ Operator เกิดขึ้นตลอดการทำงาน ส่วนการเปรียบเทียบ Precedence อาจเกิดซ้ำเมื่อมี Operator ความสำคัญเท่ากันหรือต่ำกว่าบน Stack

ประเภท	ตัวอย่าง	ผลที่คาดหวัง
Normal	3 + 4 * 2	11
Normal with parentheses	(3 + 4) * 2	14
Boundary	(((2 + 3))) * 4	20
Error	(3 + 4	Validation Error
Error	10 / (5 - 5)	ArithmeticException

Algorithm A: Infix $\rightarrow$ Postfix $\rightarrow$ Evaluation

แบ่งเป็น 2 เฟส: ใช้ **Operator Stack** แปลง Infix เป็น Postfix ตามหลัก **Shunting-Yard** แล้วใช้ **Operand Stack** ประเมิน Postfix

เฟส 1: InfixToPostfix

1. สร้าง empty operatorStack และ postfixList
2. FOR EACH token: ถ้าเป็น number $\rightarrow$ append to postfixList
3. ถ้าเป็น ($\rightarrow$ push; ถ้าเป็น) $\rightarrow$ pop จนกว่าจะเจอ (
4. ถ้าเป็น operator $\rightarrow$ pop operator ที่มี precedence $\geq$ current แล้ว push current
5. END FOR $\rightarrow$ pop operator ที่เหลือทั้งหมดลง postfixList

เฟส 2: EvaluatePostfix

1. สร้าง empty operandStack
2. FOR EACH token: ถ้าเป็น number $\rightarrow$ push integer
3. ถ้าเป็น operator $\rightarrow$ pop val2, pop val1, คำนวณ val1 op val2, push result
4. RETURN ค่าที่เหลือใน Stack

☐ **Complexity:** Time $O(n)$ / Space $O(n)$ – ข้อดีคือ Postfix สามารถเก็บไว้ใช้ซ้ำได้ ข้อจำกัดคือมีการประมวลผล 2 เฟส

Algorithm B: Direct Infix Evaluation

ประมวลผล Infix ใน **รอบเดียว** โดยใช้ Operand Stack เก็บค่าและ Operator Stack เก็บเครื่องหมาย เมื่อพบ Operator ใหม่จะประมวลผล Operator ที่มี Precedence สูงกว่าหรือเท่ากันก่อน

1

สร้าง Stack

operandStack และ operatorStack วางเปล่า

2

อ่าน Token

Number $\rightarrow$ push operand; ($\rightarrow$ push operator;) $\rightarrow$ apply จนกว่าจะเจอ (

3

จัดการ Operator

Apply operators ที่มี precedence $\geq$ current ก่อน แล้ว push operator ใหม่

4

คืนผลลัพธ์

Apply operator ที่เหลือทั้งหมด แล้ว pop ค่าสุดท้ายจาก operandStack

❏ **Procedure ApplyTopOperator:** pop op $\rightarrow$ pop val2 $\rightarrow$ pop val1 $\rightarrow$ push ExecuteOperation(val1, val2, op) – **Complexity:** Time $O(n)$ / Space $O(n)$ – Single Pass ไม่ต้องเก็บ Postfix

ตัวอย่างบังคับ: 3 + 4 * 2 / (1 - 5)

Postfix: 3 4 2 * 1 5 - / + | **ผลลัพธ์:** 1 – ทั้ง Algorithm A และ B ให้ผลลัพธ์เดียวกัน

Step	Token	Operator Stack	Postfix / Action
1	3	[]	3
2	+	[+]	3
3-5	4, *, 2	[+, *]	3, 4, 2
6	/	[+, /]	Pop * → 3, 4, 2, *
7-10	(, 1, -, 5	[+, /, (, -]	..., 1, 5
11	)	[+, /]	Pop -
12	End	[]	Pop /, + → final Postfix

❏ สำหรับ Algorithm B เมื่ออ่านนิพจน์เดียวกัน ผลลัพธ์สุดท้ายคือ 1 เช่นเดียวกับ Algorithm A

การอธิบายความถูกต้อง: Loop Invariant

Precondition

Input เป็นนิพจน์ Infix ที่ผ่าน Tokenize และ Validation แล้ว ไม่มีวงเล็บไม่สมดุq ไม่มี Operator ผิดตำแหน่ง และไม่มี Empty Expression

Postcondition

เมื่ออ่าน Token ครบ Operator Stack จะถูกประมวลผลจนว่าง และ Operand Stack เหลือค่าเดียวซึ่งเป็นผลลัพธ์ของนิพจน์ทั้งหมด

Loop Invariant ของ Algorithm B

ก่อนเริ่มรอบที่ประมวลผล Token ไปแล้ว i ตัว: **Operand Stack** เก็บผลลัพธ์ของนิพจน์ย่อยที่คำนวณเสร็จจากซ้ายไปขวาโดยไม่สูญหาย และ **Operator Stack** เก็บเฉพาะ Operator / วงเล็บเปิดที่ยังรอการประมวลผล โดยลำดับ Precedence ภายในขอบเขตวงเล็บไม่ลดลงจากล่างขึ้นบน

Maintenance

- **Number:** push เป็นนิพจน์ย่อยที่สมบูรณ์
- **(:** สร้าง Boundary ใหม่ใน Operator Stack
- **Operator:** ประมวลผล Operator ที่มี Precedence $\geq$ ก่อน แล้ว push ตัวใหม่
- **):** ประมวลผลจนถึง (และลบ Boundary นั้น

การคำนวณแบบ non-commutative รักษาลำดับซ้าย-ขวาโดยกำหนด **val2** เป็นค่าที่ pop ครั้งแรกและ **val1** เป็นค่าที่ pop ครั้งที่สอง แล้วคำนวณ **val1 op val2**

การพัฒนาโปรแกรม Java

Class	หน้าที่
group4.Main	เมนู รับ Input เรียก A/B และรับ Test Cases
group4.models.EvaluationResult	เก็บ Result, Postfix, Time และ Operation Counts
group4.utils.Tokenizer	แปลง String เป็น Token และกำหนด Precedence
group4.utils.ExpressionValidator	ตรวจ Empty, Parentheses และ Token Sequence
group4.algorithms.AlgorithmA	Infix → Postfix → Evaluation
group4.algorithms.AlgorithmB	Direct Infix ด้วย 2 Stacks
group4.ExpressionGenerator	สร้างนิพจน์สำหรับ Performance Test
group4.ExperimentRunner	รับ n = 100, 1,000, 10,000, 50,000 อย่างละ 5 รอบ

Error Handling

- Invalid character → `IllegalArgumentException`
- Unbalanced parentheses → Validation Error
- Operator sequence ผิด → Validation Error
- Division by zero → `ArithmeticException`
- โปรแกรมไม่ Crash และกลับสู่เมนูหลัก

โครงสร้าง Repository

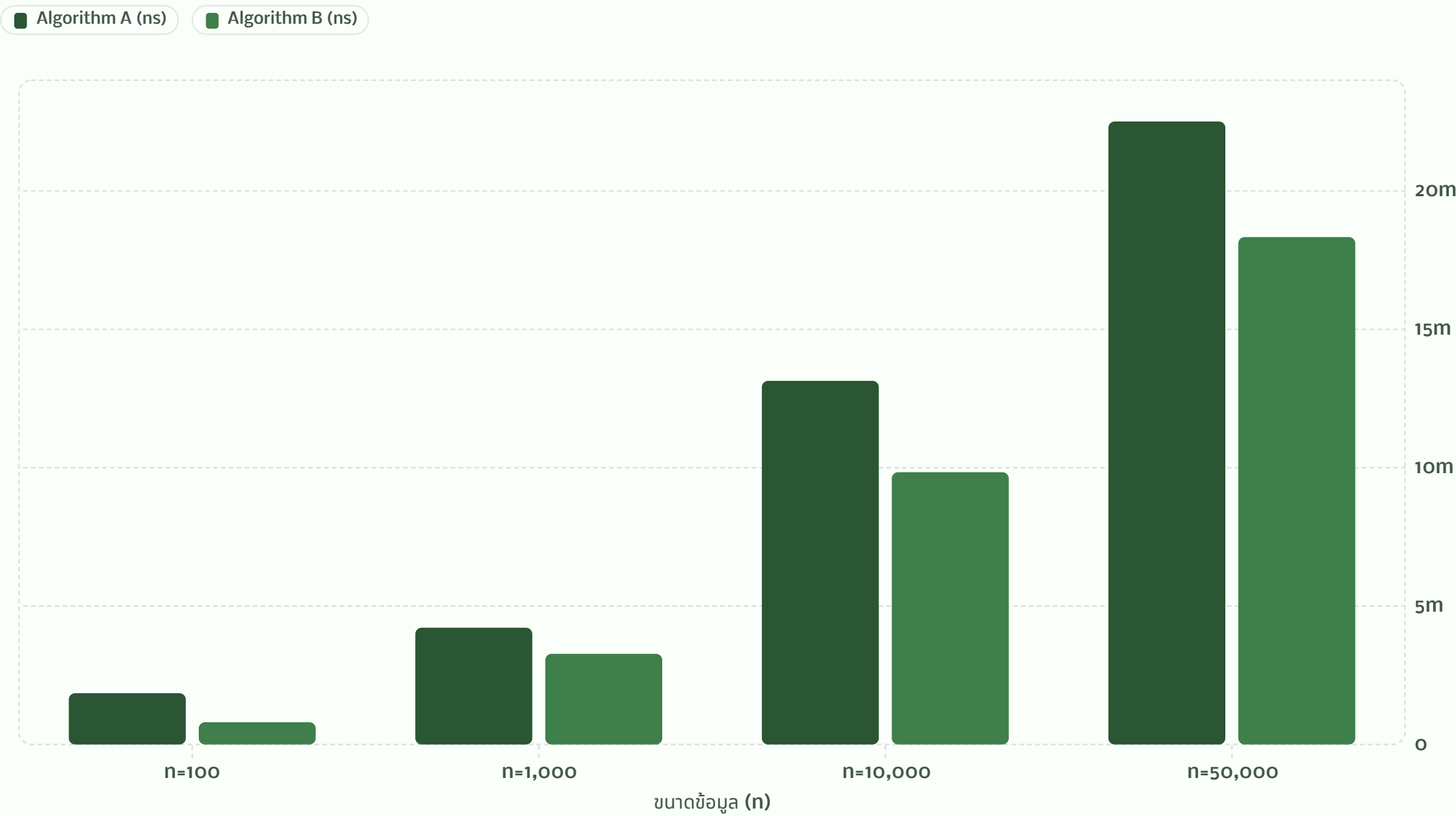
- `src/` – Main, algorithms/, models/, utils/
- `test/` – test_cases.csv
- `results/` – experiment_results.csv
- `diagrams/` – flowcharts, class diagram, performance graph
- `README.md`, `AI_Usage_Log.md`

ผลการทดสอบและการทดลองประสิทธิภาพ

ผลการทดสอบ 8 Mandatory Test Cases

ID	Input	Expected	Actual	Status
TCo1	3 + 4 * 2	Result = 11	Result = 11	✓ PASSED
TCo2	(3 + 4) * 2	Result = 14	Result = 14	✓ PASSED
TCo3	((8 + 2) * 5)	Result = 50	Result = 50	✓ PASSED
TCo4	3 + 4	Error: Missing ')'	Validation Error Caught	✓ PASSED
TCo5	3 + 4)	Error: Extra ')'	Validation Error Caught	✓ PASSED
TCo6	3 + * 4	Error: Invalid operator sequence	Validation Error Caught	✓ PASSED
TCo7	10 / (5 - 5)	Error: Division by zero	ArithmeticException Caught	✓ PASSED
TCo8		Error: Expression is empty	Validation Error Caught	✓ PASSED

ผลการทดลอง Performance (n = 100 ถึง 50,000)



Algorithm B เร็วกว่า Algorithm A ทุกขนาดที่ทดสอบ แม้ทั้งคู่มี $O(n)$ – ตัวอย่าง $n=50,000$: $A \approx 22.5\text{ ms}$, $B \approx 18.3\text{ ms}$ ความแตกต่างเกิดจาก **Constant Factor**: Algorithm A มี 2 เฟสและต้องสร้าง Postfix List ขณะที่ Algorithm B ประมวลผลแบบ Single Pass จำนวน Push/Pop เพิ่มขึ้นตามขนาดข้อมูลอย่างเป็นเชิงเส้น สอดคล้องกับ $O(n)$

สรุปผล การเปรียบเทียบ และสิ่งที่ได้เรียนรู้

ประเด็น	Algorithm A	Algorithm B
แนวทาง	Infix → Postfix → Evaluate	Direct Infix
Time Complexity	$O(n)$	$O(n)$
Space Complexity	$O(n)$	$O(n)$
Postfix ใช้ซ้ำ	ได้	ไม่ได้โดยตรง
ความเร็วในการทดลอง	ช้ากว่า	เร็วกว่า

วิธีที่เหมาะสมกว่า

สำหรับการคำนวณผลลัพธ์เพียงครั้งเดียว **Algorithm B** เหมาะกว่า เพราะผลการทดลองแสดงเวลาต่ำกว่าและไม่ต้องสร้าง Postfix List หากต้องเก็บ Postfix เพื่อใช้ซ้ำหรือส่งต่อให้ระบบอื่น **Algorithm A** มีประโยชน์มากกว่า

ข้อจำกัดและแนวทางพัฒนา

- ยังไม่รองรับทศนิยม, Unary Minus, ตัวแปร และฟังก์ชันทางคณิตศาสตร์
- สามารถพัฒนาต่อด้วย %, ^ และ Associativity ของ ^
- เพิ่ม [] และ {}
- เพิ่มการแสดงผล Trace ของ Stack แบบละเอียดในโปรแกรม

สิ่งที่ได้เรียนรู้

เห็นความแตกต่างระหว่าง **Asymptotic Complexity** กับประสิทธิภาพจริงจาก Constant Factor และได้ฝึก Stack, Amortized Analysis, Loop Invariant, Testing และ Performance Measurement

 **Generative AI Usage:** AI ใช้เพื่อช่วยเรียนรู้และร่างแนวทางเท่านั้น – ผลลัพธ์ทุกส่วนต้องถูกตรวจสอบและจัดทำอธิบาย Source Code ได้ กลุ่มแก้ไข Tokenizer (เขียนแบบอ่าน char ทีละตัว), แก้ Big-O เป็น $O(n)$ ด้วย Amortized Analysis, เพิ่ม boundary/error cases ใน Test Cases และใช้ AI ช่วยร่างเครื่องมือวัดผลโดยไม่เปลี่ยน Logic A/B